

Detailed Info of TryHackMe Room

Blue

MS17-010 (EternalBlue) · Exploits & Payloads · Meterpreter ·
Process Migration · Hashdump & Password Cracking · Post-Exploitation

*A structured walkthrough of the vulnerability, exploitation chain,
and post-exploitation workflow covered in the TryHackMe "Blue" room.*

Table of Contents

Part 1 — Understanding the Vulnerability & Exploit Chain

1. Understanding MS17-010
2. Finding the Exploit
3. Exploit vs Payload
4. Payloads Used
5. Backgrounding the Shell
6. Upgrading to Meterpreter
7. The SESSION Variable
8. When Meterpreter Is Actually Transferred
9. exploit/multi/handler

Part 2 — Meterpreter, Migration & Post-Exploitation

1. Using Meterpreter
2. Process Enumeration
3. What Is a Stable Process?
4. Process Migration (Low-Level)
5. Does the Old Process Disappear?
6. Understanding Hashdump
7. Default vs Non-Default Users
8. Guest Account
9. LM vs NTLM Hashes
10. Password Cracking (Offline)
11. Flag Hints
12. Payload vs Post-Exploitation Module

PART 1: Understanding the Vulnerability & Exploit Chain

1. Understanding MS17-010

What is MS17-010?

MS17-010 is a Remote Code Execution (RCE) vulnerability in Microsoft's implementation of SMBv1 (Server Message Block version 1).

SMB is the protocol Windows uses for:

- File sharing
- Printer sharing
- Network resource access

Why is it vulnerable?

Normally, the SMB service receives network requests and processes them. The vulnerability exists because SMBv1 does not properly validate certain specially crafted SMB packets.

- An attacker can send a malicious SMB packet containing carefully crafted values.
- Instead of rejecting it, Windows incorrectly processes it.
- This causes memory corruption.

What memory gets corrupted?

The corrupted memory belongs to the running SMB service itself. Specifically: the SMB service is running on the victim, Windows allocates RAM for that service, and the malicious packet causes the SMB service to overwrite its own memory.

Initially it does **not** corrupt Explorer, Chrome, or other Windows services — it corrupts the memory of the vulnerable SMB service.

Why is memory corruption dangerous?

Memory stores variables, program instructions, function addresses, and execution flow. By carefully corrupting memory, the attacker can change where the program executes next — instead of continuing Windows code, execution is redirected to attacker-controlled code. This allows Remote Code Execution.

Was the packet obviously malicious?

No. The SMB packet still looked like a legitimate SMB request. It did not contain anything saying "this is an attack." The attack came from specially crafted field values and abusing a programming mistake inside SMB.

Security tools that know EternalBlue can detect it, but to the vulnerable SMB service it initially looked like a normal SMB request.

Best way to understand EternalBlue

It does **not** permanently modify Windows. Instead, EternalBlue temporarily hijacks the running SMB service in RAM — meaning SMB keeps running, but it now executes attacker-controlled code. The program on disk is not modified; the hijack exists only while the process is running.

2. Finding the Exploit

Enumeration only tells us *"the machine is vulnerable"* — it does not tell us how to exploit it.

Inside Metasploit:

```
search ms17-010
```

This searches Metasploit's database for exploit modules. Think of Metasploit as a toolbox — we are asking: *"Show me every tool capable of exploiting MS17-010."*

Why EternalBlue?

```
exploit/windows/smb/ms17_010_eternalblue
```

This exploit already knows which SMB packets to send, their order, their structure, and how to trigger the vulnerability. It is simply an automated implementation of the EternalBlue attack.

3. Exploit vs Payload

This is one of the most important concepts in Metasploit.

Exploit

Purpose: Gets code execution. Its only job is to trigger the vulnerability and hijack the vulnerable service. Once code execution is achieved, the exploit's job is finished.

Payload

Purpose: Decides what code the victim executes after exploitation succeeds. The exploit gets us inside; the payload decides what happens next.

Exploit: "I know how to unlock the broken door."

Payload: "Now that I'm inside, what should I do?"

Examples: open cmd.exe, run Meterpreter, execute another program, create another connection.

Very important: *Exploit ≠ Payload. The exploit only creates the opportunity to execute code. The payload is the code that gets executed.*

4. Payloads Used

Shell Payload

```
windows/x64/shell/reverse_tcp
```

The victim executes code that launches cmd.exe and connects back to Kali. Result:

```
C:\Windows\system32>
```

This is simply a normal Windows command prompt. You can run:

```
dir  
cd  
type  
whoami
```

Meterpreter Payload

```
windows/x64/meterpreter/reverse_tcp
```

The victim executes the Meterpreter program directly in RAM. Instead of cmd.exe, you receive:

```
meterpreter >
```

Meterpreter includes many built-in capabilities:

- ps
- getuid
- migrate
- hashdump
- upload
- download

Meterpreter is much more powerful than a basic shell.

Why did the room use Shell first?

The room intentionally avoided Meterpreter initially, in order to teach the full progression: exploit → shell → shell upgrade → Meterpreter — instead of immediately giving Meterpreter.

5. Backgrounding the Shell

After exploitation we were inside C:\Windows\system32>. We pressed:

```
CTRL + Z
```

Confirmed:

```
y
```

What happened?

The shell was **not** terminated. Instead, Metasploit detached from it and stored it in the session list. Think of it like putting a phone call on hold — the connection remains alive. You simply return to:

```
msf6 >
```

The shell is still available. Example:

```
sessions  
  
1 shell x64/windows
```

We can reconnect anytime:

```
sessions -i 1
```

The shell remains available until the victim disconnects, the session dies, we kill it, or Metasploit exits.

6. Upgrading to Meterpreter

Initially we had:

Session 1

.l.

cmd.exe

This shell is limited — it cannot perform advanced Meterpreter operations.

Why upgrade?

We wanted Meterpreter-only features:

- ps
- hashdump
- migrate
- getuid
- upload
- download

What does upgrading actually do?

Metasploit uses the existing shell to transfer the Meterpreter payload and execute it on the victim. The victim runs the Meterpreter payload in RAM, and the payload creates a new reverse connection back to Kali. That becomes Session 2.

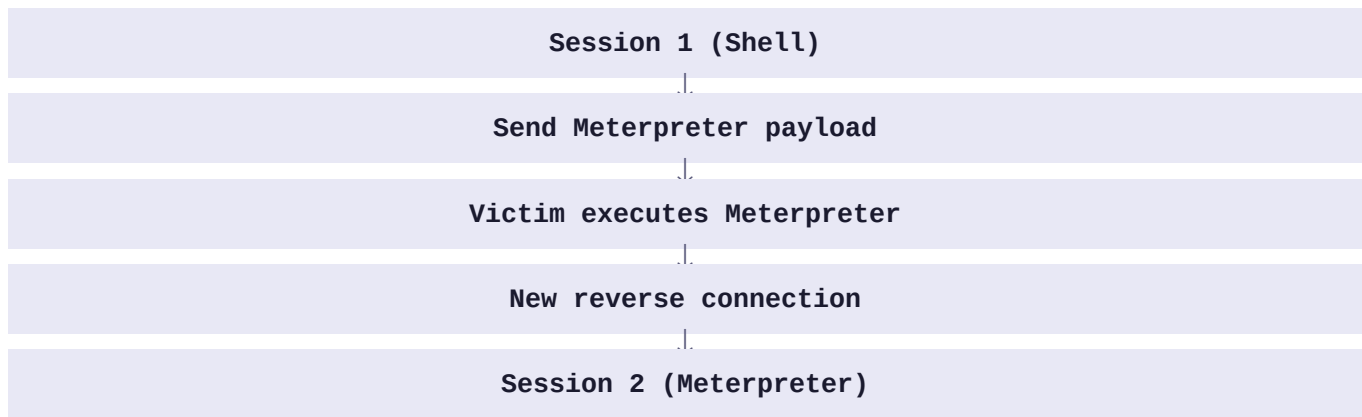
Very important: the original shell is not replaced. After upgrading we have both Session 1 (Shell) and Session 2 (Meterpreter) running side by side.

7. The SESSION Variable — set SESSION 1

The upgrade module needs to know: "Which existing shell should I use?"

```
set SESSION 1
```

This means: use shell session 1 to send and execute the Meterpreter payload. This is the correct mental model. Flow:



8. When Was Meterpreter Actually Transferred?

Not while typing inside `C:\Windows\system32>`. Instead, after backgrounding the shell:

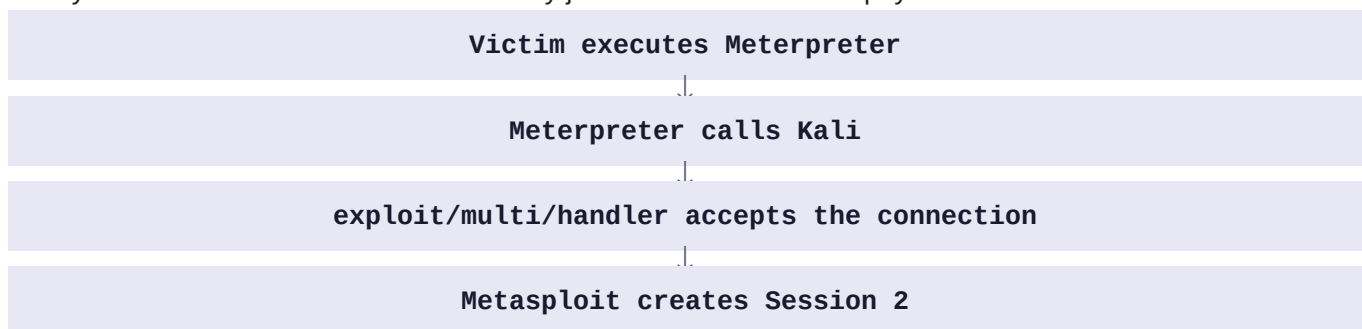
```
use post/multi/manage/shell_to_meterpreter
set SESSION 1
run
```

The important command is **run**. At this moment:

- Metasploit connects to Session 1
- Transfers the Meterpreter payload
- Victim executes it
- Meterpreter starts
- Meterpreter connects back
- Session 2 is created

9. exploit/multi/handler

Very short definition: it is a **listener**. Its only job is to wait for reverse payloads to connect back. Flow:



Think of it as someone waiting for an incoming phone call.

Key Takeaways

- ✓ MS17-010 is an SMBv1 Remote Code Execution vulnerability.
- ✓ The bug allows specially crafted SMB packets to corrupt the SMB service's memory.

- ✓ EternalBlue temporarily hijacks the SMB service in RAM.
- ✓ Exploit = gains code execution. Payload = decides what code executes after exploitation.
- ✓ shell/reverse_tcp gives a normal Windows cmd.exe.
- ✓ meterpreter/reverse_tcp gives a powerful Meterpreter session.
- ✓ Backgrounding a shell (Ctrl + Z) keeps the shell alive while returning to the Metasploit prompt.
- ✓ set SESSION 1 tells the upgrade module to use the existing shell connection.
- ✓ Running shell_to_meterpreter transfers the Meterpreter payload through the shell, the victim executes it, and a new Meterpreter session is created.
- ✓ exploit/multi/handler listens for the reverse connection from the Meterpreter payload and turns it into a usable Metasploit session.

PART 2: Meterpreter, Migration & Post-Exploitation

1. Using Meterpreter

After upgrading the basic shell, we interacted with the Meterpreter session:

```
sessions -i 2
```

Meaning: **sessions** shows or interacts with Metasploit sessions, **-i** means interact, and **2** is the session ID. This simply tells Metasploit to connect us to Meterpreter session 2.

Verifying Our Privileges

```
getuid
```

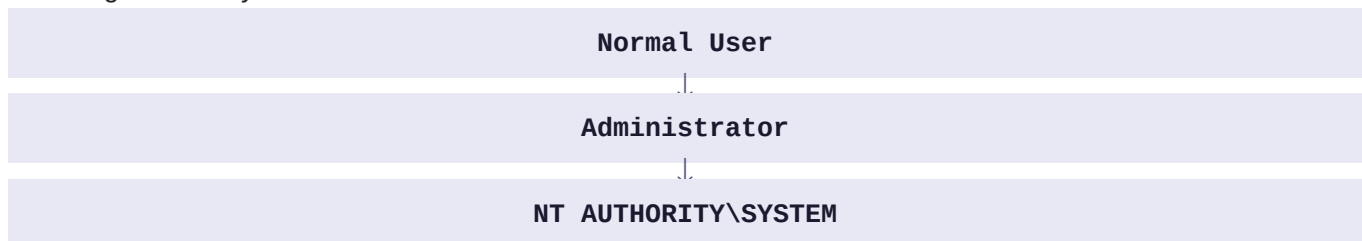
Output:

```
NT AUTHORITY\SYSTEM
```

This asks Windows: "Which user is this Meterpreter process currently running as?" It only displays our current privileges.

What is NT AUTHORITY\SYSTEM?

Privilege hierarchy:



SYSTEM is a special Windows account used by the operating system itself. It has the highest local privileges on the machine.

We already obtained SYSTEM because EternalBlue exploited the SMB service, which itself runs as SYSTEM. Therefore our payload also executed as SYSTEM.

2. Process Enumeration

```
ps
```

Displays every running process on the compromised Windows machine. Important columns:

- **PID** — Process ID
- **PPID** — Parent Process ID

- **Name** — Process name
- **User** — Which account the process runs under

Example:

PID	Name	User
1308	spoolsv.exe	NT AUTHORITY\SYSTEM

Why did we run **ps**? Not simply to list processes — the real objective was to find a stable SYSTEM process to migrate into.

3. What Is a Stable Process?

A stable process is simply a process that normally stays running for the entire lifetime of Windows.

Stable

- spoolsv.exe
- services.exe
- lsass.exe
- winlogon.exe

Unstable

- notepad.exe
- chrome.exe
- cmd.exe

Those user applications can be closed at any time.

How do we know a process is stable?

You cannot determine it just by looking once — you learn through Windows knowledge. General rule: if it is a Windows service, running as SYSTEM, starts automatically, and rarely exits, then it is usually a good migration target.

Why did we choose spoolsv.exe?

- It is the Print Spooler service.
- Runs as SYSTEM.
- Starts automatically.
- Normally remains running until shutdown.

Therefore it is a safe place for Meterpreter to live.

4. Process Migration (Low-Level)

```
migrate 1308
```

Low-level process, step by step:

Step 1

Meterpreter opens the target process using Windows APIs. Windows returns a process handle.

Step 2

Meterpreter allocates memory inside the target process. Think of this as creating empty space inside spoolsv.exe.

Step 3

Meterpreter copies its own code into that allocated memory. Now spoolsv.exe contains Windows code and Meterpreter code side by side.

Step 4

Meterpreter creates a new thread inside spoolsv.exe. A thread is simply another path of execution inside the process. Now spoolsv.exe continues running normally while also executing Meterpreter.

Step 5

Once the new Meterpreter confirms it works, the old Meterpreter exits and the new Meterpreter becomes the active session.

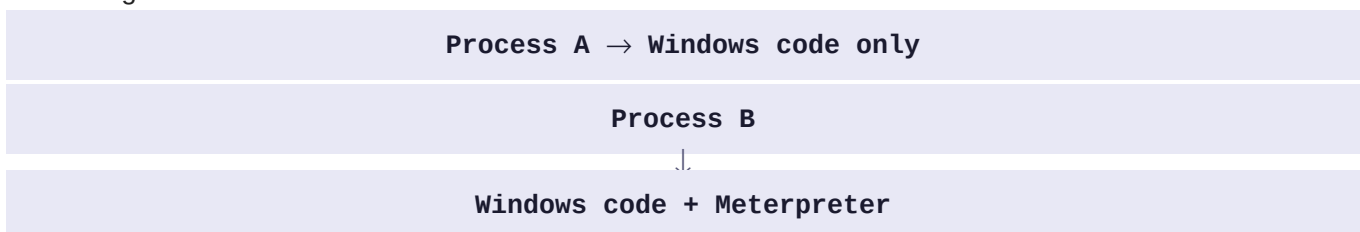
Important clarification: migration is not moving the actual bytes from one process to another. Instead: create a new copy, start executing it, terminate the old one. From our perspective it behaves exactly like a move.

5. Does the Old Process Disappear?

No. Example — before migration:



After migration:



The original Windows process keeps running normally. Only the Meterpreter code exits.

Is the Meterpreter code still inside the old process?

Normally no. During migration both copies briefly exist; once migration succeeds, the old Meterpreter exits and its allocated memory is released. The memory contents may physically remain in RAM until overwritten, but there is no longer a running Meterpreter inside that process.

Can we migrate back?

Yes — if the original process still exists, Meterpreter simply repeats the migration process: allocate memory, copy itself, execute, remove the current instance. It does **not** reconnect to the old Meterpreter; it creates a completely new one.

6. Understanding Hashdump

hashdump

Purpose: extract password hashes from the Windows SAM database. Typical output:

```
Administrator:500:LM:NTLM
Guest:501:LM:NTLM
Jon:1000:LM:NTLM
```

Fields: Username, RID, LM Hash, NTLM Hash.

7. Default vs Non-Default Users

Windows automatically creates **Administrator** and **Guest** — these are default accounts. Any user created after Windows installation is considered a non-default account.

Example: Administrator, Guest, **Jon** — Jon is the non-default account, meaning he represents the actual human user of the computer.

This is valuable because user accounts usually contain:

- Documents
 - Browser data
 - Passwords
 - SSH keys
 - Work files
 - Personal information
-

8. Guest Account

Purpose: a temporary account for someone to use the computer without creating a personal account.

- Very limited permissions.
 - Built into Windows.
 - Disabled by default on modern Windows for security reasons.
-

9. LM vs NTLM Hashes

Hashdump retrieves both, in the format:

```
Username : RID : LM Hash : NTLM Hash
```

However, we only cared about the NTLM hash. Reason: LM is an old legacy hashing algorithm, while NTLM is what Windows actually uses today. Modern Windows normally disables LM hash storage because it is weak.

Why did Windows ever store LM hashes?

Backward compatibility — older Windows systems only understood LM hashes. To support both old and new systems, Windows originally stored both an LM hash and an NTLM hash for every password. Today, LM is usually disabled and NTLM is the important hash.

10. Password Cracking (Offline)

We extracted Jon's NTLM hash and saved it:

```
echo ' ' > hash.txt
```

Then used:

```
john --format=NT hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
```

How does John the Ripper work?

It does **not** reverse hashes. Instead, for every candidate password it generates an NTLM hash and compares it with the stolen NTLM hash. If the hashes match, the correct password has been found.

Why is this called Offline Cracking?

After stealing the hash, the victim machine is no longer involved — everything happens on our Kali machine.

- No login attempts.
- No network traffic.
- No account lockouts.
- Much harder to detect.

What password did we recover?

We recovered Jon's actual Windows login password:

```
alqfna22
```

11. Flag Hints

Flag 1

Hint: *"Found at the system root."* Meaning C:\ — so we checked:

```
C:\flag1.txt
```

Flag 2

Hint: "Found in the Windows configuration area." Meaning:

```
C:\Windows\System32\config\
```

This directory contains important Windows configuration files including the SAM database.

Flag 3

Hint: "Administrators usually have pretty interesting things saved."

Important clarification: Jon was not necessarily an Administrator. The hint is general advice — real penetration testers commonly inspect user folders because they often contain valuable information. The room simply chose C:\Users\Jon\Documents\ to demonstrate this concept.

12. Meterpreter Payload vs Post-Exploitation Module

This was an important distinction.

Payload

Examples:

```
windows/x64/shell/reverse_tcp  
windows/x64/meterpreter/reverse_tcp
```

Payloads execute immediately after exploitation.

In Our Lab

We intentionally used:

```
windows/x64/shell/reverse_tcp
```

This gave us a basic Windows shell. Then we used:

```
post/multi/manage/shell_to_meterpreter
```

This is **not** a payload — it is a Post-Exploitation Module. Its purpose is to upgrade an existing shell into a Meterpreter session. Internally it launches a Meterpreter payload through the existing shell and establishes a new Meterpreter connection.

If We Had Selected the Meterpreter Payload Initially

If we had used windows/x64/meterpreter/reverse_tcp, the workflow would have been:

Exploit

↓

Meterpreter Payload

↓

Meterpreter Session

We would have skipped the shell upgrade completely. The room intentionally chose the basic shell first to teach the difference between Exploit, Payload, and Post-Exploitation Module.

Key Takeaways

- ✓ Meterpreter is a powerful post-exploitation environment.
- ✓ Process migration injects Meterpreter into another process for stability.
- ✓ Stable Windows services are preferred migration targets.
- ✓ Migration copies Meterpreter into a new process, starts it there, then terminates the old instance.
- ✓ hashdump extracts password hashes, not plaintext passwords.
- ✓ John the Ripper generates hashes from candidate passwords and compares them with the stolen hash.
- ✓ Offline cracking occurs entirely on the attacker's machine after the hash has been stolen.
- ✓ Non-default accounts usually represent real users and are often the most valuable targets during post-exploitation.
- ✓ A payload determines the initial access you receive, while a post-exploitation module enhances or upgrades that access after compromise.